

PROGRAMACIÓN ORIENTADA A OBJETOS POLIMORFISMO

Curso de Programación de Algoritmos con Java
Universidad Técnica Particular de Loja – Ecuador
Wayner Bustamante – wxbustamante@utpl.edu.ec

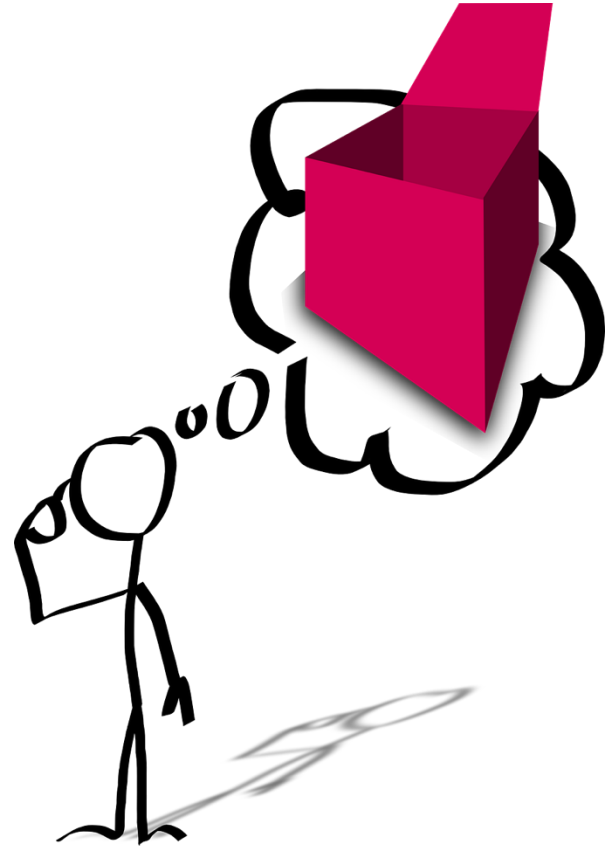


AGENDA

- ✓ Pilares fundamentales de la POO
- ✓ Que es Polimorfismo
- ✓ Ejemplos polimorficos
- ✓ Clases y métodos abstractos
- ✓ Estudio de casos *(ejercicios)*

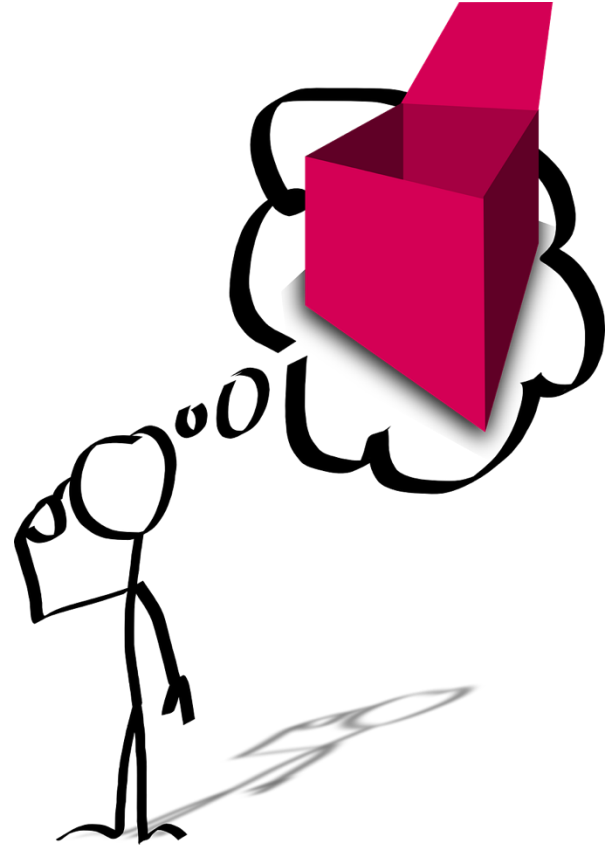
ENCAPSULAMIENTO

- ✎ Unir en Clases, características (*atributos*) y comportamientos (*métodos*).
- ✎ Es tener todo en una sola entidad -> **Clase**.
- ✎ Util por que facilita manejar la complejidad, de las clases como cajas negras, donde sólo se conoce el comportamiento, pero no los detalles internos (*implementación*).
- ✎ Interesa conocer qué hace una **Clase** pero no cómo lo hace.
- ✎ Agrupa en un ente información y funcionalidad.



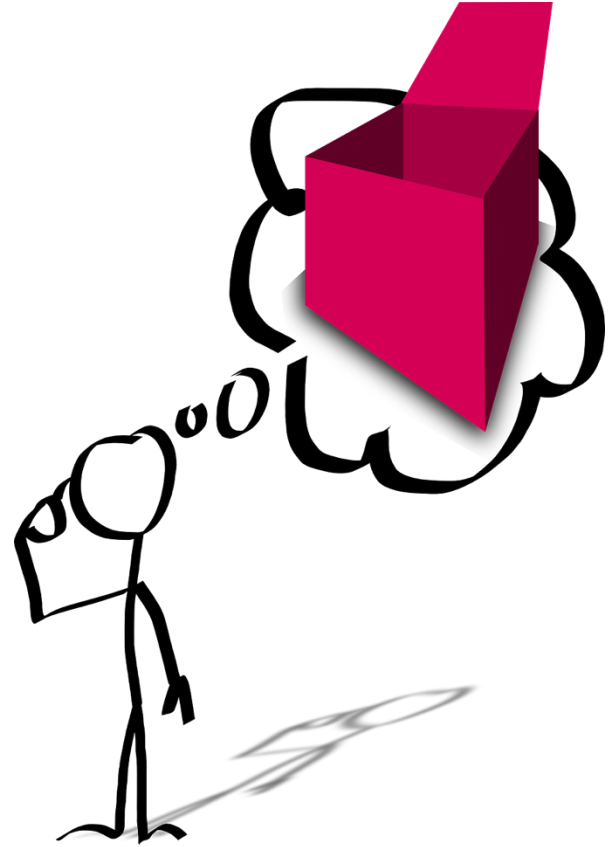
ABSTRACCIÓN

- ✎ Representación de las características esenciales de algo sin incluir antecedentes o detalles irrelevantes.
- ✎ Alejar, sustraer, separar: aislar conceptualmente una propiedad(es)/función(es) concreta(s) de un objeto, para pensar en qué es.
- ✎ Cada objeto sirve como modelo de un "agente" abstracto que puede realizar trabajo, informar y cambiar su estado y "comunicarse" con otros objetos sin revelar cómo se implementan estas características.



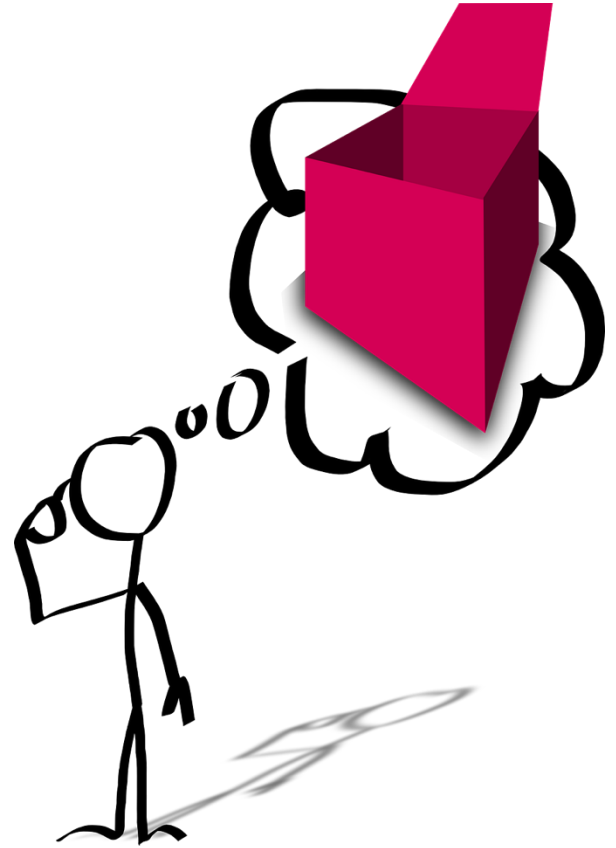
HERENCIA

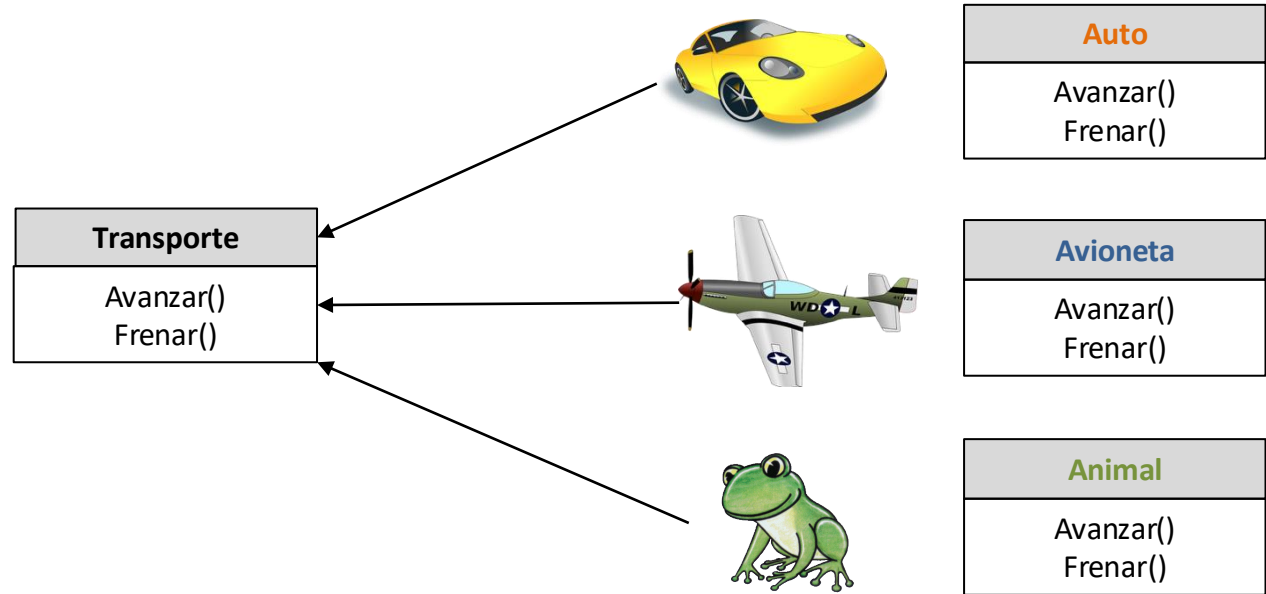
- ✎ Mecanismo para compartir automáticamente métodos y atributos entre clases y subclases.
- ✎ Forma de reutilizar software creando nuevas clases (*subClases*) a partir de otras (*superClases*), absorbiendo, modificando y mejorando los miembros y capacidades de sus clases padre.
- ✎ Especializa o extiende funcionalidad.
- ✎ Optimiza cantidad de código.
- ✎ Permite diseño jerarquico de clases.



POLIMORFISMO

- ✎ Que puede adoptar múltiples formas.
Propiedad capaz de atravesar numerosos estados.
- ✎ Permite “programar en forma general”, en vez de “forma específica”.
- ✎ Confía en que cada objeto sepa como “hacer lo correcto”.
- ✎ Hace posible enviar mensajes sintácticamente iguales a objetos de tipos distintos.
- ✎ Comportamiento de los objetos, no a su pertenencia a una jerarquía de clases (tipos de datos).
- ✎ "Polimorfismo" = muchas formas.

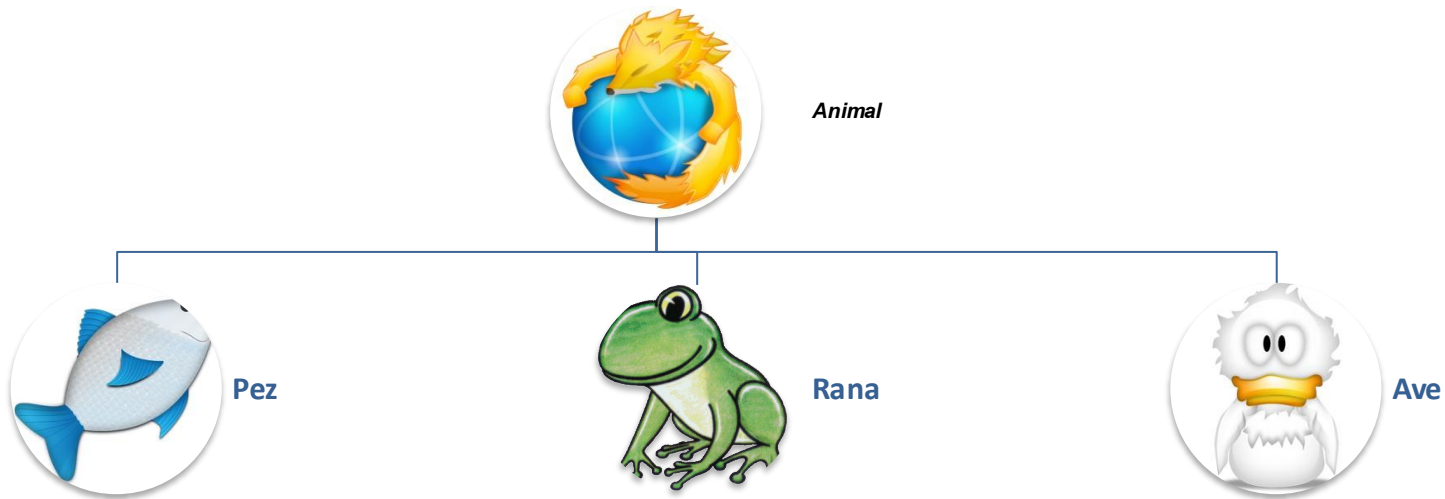




Polimorfismo

"hacer algo de muchas formas"

Auto ➡ Avanzar(), Frenar()
Avioneta ➡ Avanzar(), Frenar()
Animal ➡ Avanzar(), Frenar()



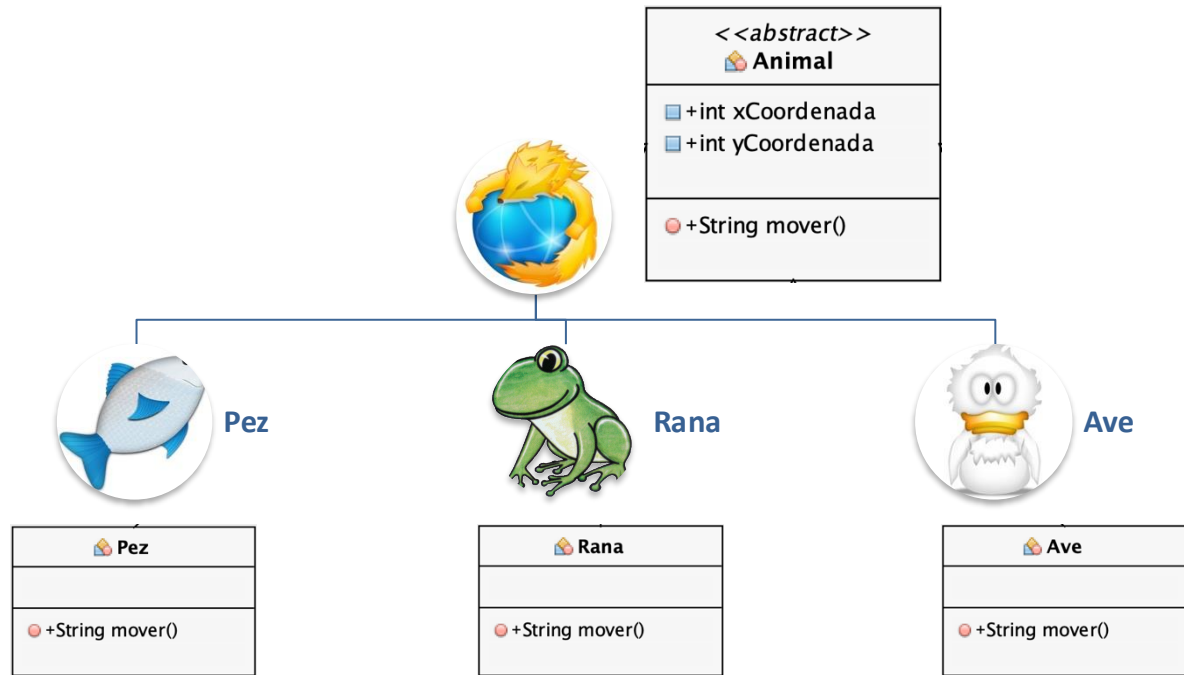
Polimorfismo

Animal 📌 Clase abstracta. Comportamiento general

Pez 📌 Clase concreta. Comportamiento específico

Rana 📌 Clase concreta. Comportamiento específico

Ave 📌 Clase concreta. Comportamiento específico



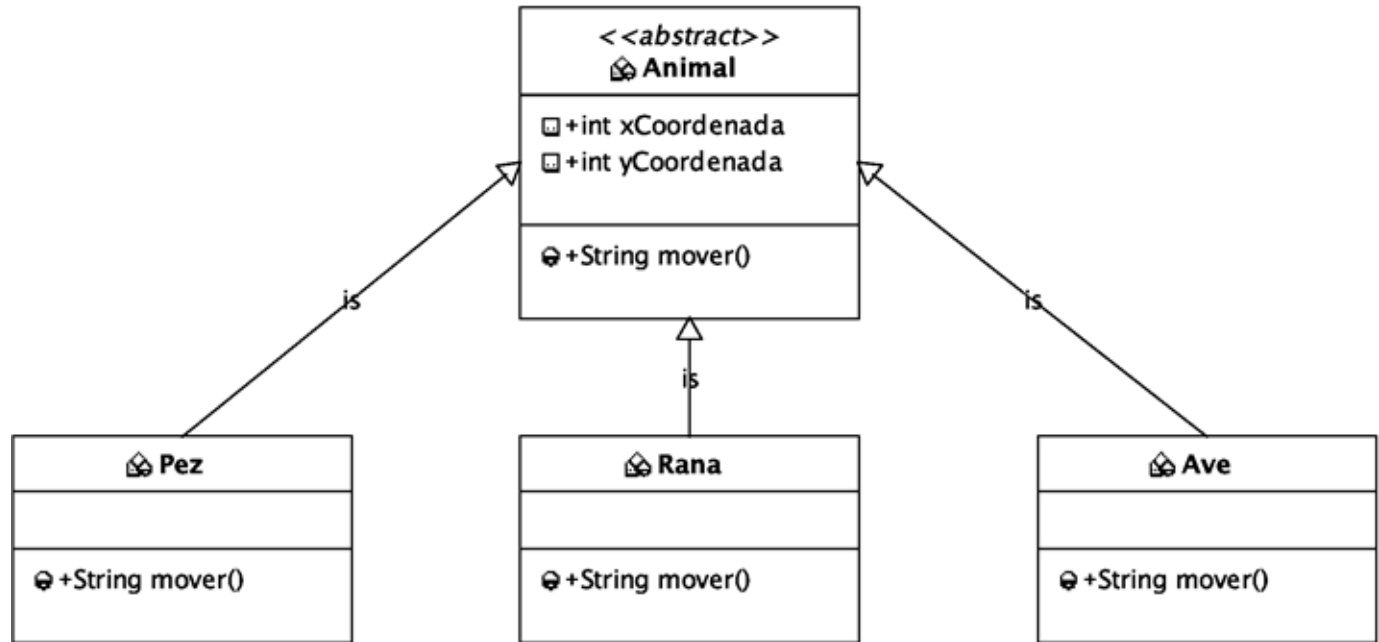
Polimorfismo

Animal ➡ Clase abstracta. Comportamiento general

Pez ➡ Clase concreta. Comportamiento específico

Rana ➡ Clase concreta. Comportamiento específico

Ave ➡ Clase concreta. Comportamiento específico



Polimorfismo

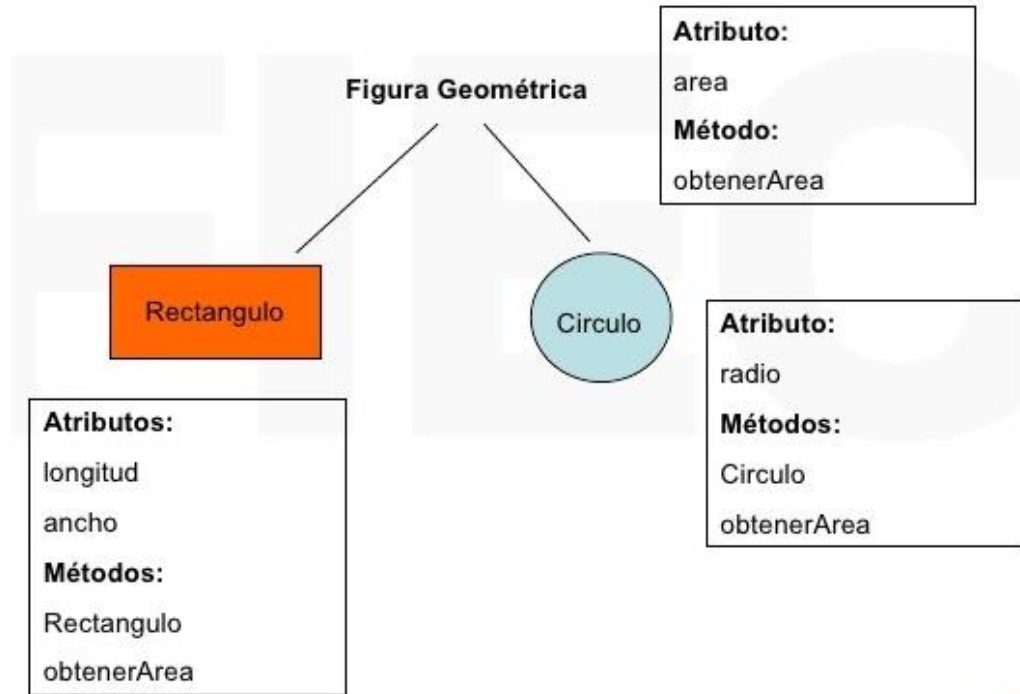
Animal ➡ Clase abstracta. Comportamiento general

Pez ➡ Clase concreta. Comportamiento específico

Rana ➡ Clase concreta. Comportamiento específico

Ave ➡ Clase concreta. Comportamiento específico

Polimorfismo



"hacer algo de muchas formas"

Figura ➡ [obtenerArea\(\)](#)
Rectangulo ➡ [obtenerArea\(\)](#)
Circulo ➡ [obtenerArea\(\)](#)

Polimorfismo

Declaro la función:

```
function estacionar( Vehiculo ) { }
```

Invoco la función: (soporto polimorfismo)

```
estacionar( Coche ) ;
```

```
estacionar( Moto ) ;
```

```
estacionar( Bus ) ;
```

No puedo invocar la función: (no lo permitiría, porque no ser clasificación de herencia de vehículos)

```
estacionar( Mono ) ;
```

```
estacionar( INT ) ;
```

En el futuro si podría: (Si creo las clases "Van" o "Nave espacial" y heredan de Vehículo)

```
estacionar( Van ) ;
```

```
estacionar( Nave espacial ) ;
```

ESTUDIO DE CASOS (1)

Las clases Pez, Rana y Ave representan los tres tipos de animales bajo investigación. Imagine que cada una de estas clases extiende a la superclase Animal, la cual contiene un método llamado mover y mantiene la posición actual de un animal, en forma de coordenadas x-y. Cada subclase implementa el método mover.



```

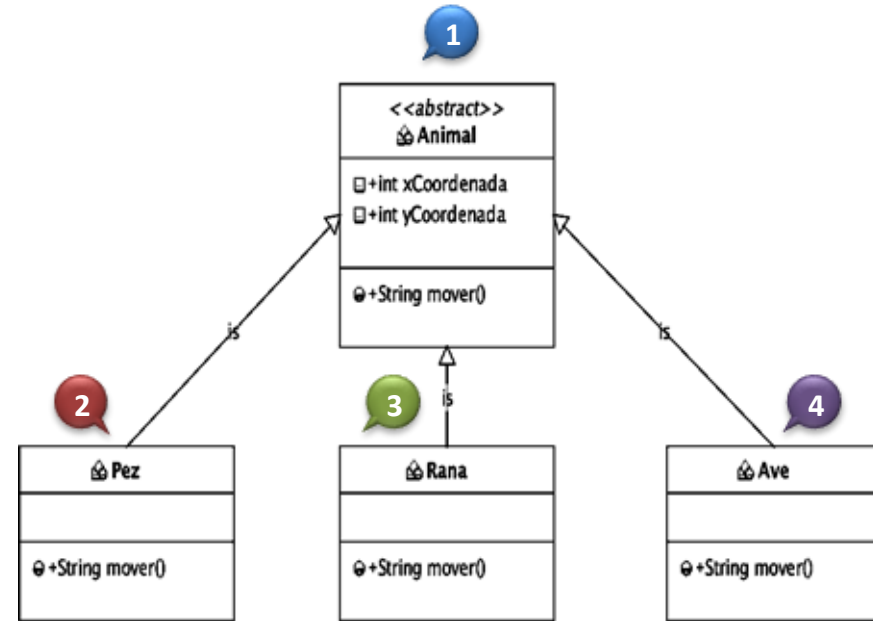
1 public abstract class Animal {
    public int xCoordenada;
    public int yCoordenada;
    public abstract String mover();
}

2 class Pez extends Animal {
    public String mover() {
        return "Pez moviendo";
    }
}

3 class Rana extends Animal {
    public String mover() {
        return "Rana moviendo";
    }
}

4 class Ave extends Animal {
    public String mover() {
        return "Ave moviendo";
    }
}

```



5

```
class TestAnimal{  
    public static ArrayList<Animal> animales = new ArrayList<Animal>();  
    public static void main(String[] args) {  
        Animal pez = new Pez();  
        Animal rana = new Rana();  
        Animal ave = new Ave();  
        animales.add(pez);  
        animales.add(rana);  
        animales.add(ave);  
        for (Animal animal : animales) {  
            System.out.println(animal.mover());  
        }  
    }  
}
```

Output - NomProyecto (run)

```
run:  
Pez moviendo  
Rana moviendo  
Ave moviendo  
BUILD SUCCESSFUL (total time: 0 seconds)
```





PROGRAMACIÓN ORIENTADA A OBJETOS POLIMORFISMO

curso de programación de algoritmos

@utpl, @wxbustamante